

LedgerTrail — Project Explainer

Everything you need to know to defend this project in front of judges

A working document, updated as the build progresses. Current status: Days 1-3 complete and verified, including the full API layer. Cursor/frontend work starts Day 4.

1. The Problem, In Your Own Words

When a Razorpay merchant gets paid, they don't get one payment per order. They get **one lump-sum bank credit per settlement cycle** — a number that already has refunds subtracted, Razorpay's fees deducted, and tax withheld. To know whether that lump sum is actually *correct*, someone has to manually check it against:

What Razorpay says it paid out (the settlement report)

What actually landed in the bank (the bank statement)

What the merchant's own records say *should* have happened (their order/refund data)

Today, this is done by hand in spreadsheets. It's slow, and — more importantly — small discrepancies (₹42 here, ₹200 there) usually get written off as "rounding" because tracing them by hand isn't worth anyone's time. That's a silent problem: unexplained gaps quietly turn into misstated financial records, and nobody notices until it's a much bigger number.

The core idea we're building around: reconciliation should be treated as a claim that must be *proven*, not assumed. If a settlement is marked "reconciled," there should be evidence for every rupee — not just a checkbox someone clicked.

2. Our Solution, In One Sentence

LedgerTrail ingests three data sources (settlement file, bank statement, order records), builds a **gross-to-net bridge** for each settlement batch ($\text{Gross} - \text{Refunds} - \text{Fees} - \text{Tax} = \text{Net}$), checks that bridge against what actually landed in the bank, and — when it doesn't match — isolates the exact gap, classifies why, and puts it in front of a human to approve. Nothing is auto-resolved. Nothing is auto-posted.

The one sentence to defend hardest under questioning

The AI never touches the arithmetic. All matching, all bridge math, all exception classification is deterministic Python — plain if/else logic and arithmetic, no model calls anywhere near it. The AI (Google Gemini) only ever explains facts that the deterministic code already computed, and it only ever writes to the *narrative text* of an approval brief — never to the database tables that hold the actual numbers.

Note on tooling vs. product: LedgerTrail was built using Claude Code (backend) and Cursor (frontend) as development tools — that's how the code was written. The AI feature *inside the running product* (the exception explanation layer) is a separate thing entirely, and uses Google's Gemini API. If asked "does your

product use Claude," the accurate answer is: development tooling, yes; the in-app AI explanation feature, no — that's Gemini. Don't conflate the two.

If a judge asks "does the AI ever calculate anything," the honest answer is always **no**, and you should be able to open `app/matching.py` or `app/bridge.py` and show them plain Python with no API calls in it.

3. Why This Architecture, Specifically

We split the system into four layers, each with exactly one job:

Layer	Job	Why it's separate
Verified data	Establishes facts (settlement rows, bank rows, order records)	If the input data is wrong, nothing downstream can be trusted — so this layer just stores facts, no interpretation
Deterministic code	Matching, bridge math, exception classification	This is the credibility layer. It has to be simple enough that anyone can read the code and verify it's correct by hand
AI (LLM)	Explains an already-computed fact packet in plain language; proposes a next action <i>from a fixed playbook</i> , never invented	AI is good at language, bad at being trusted with money math. We only use it where its weakness (occasionally being wrong or overconfident) can't cause financial harm
Human	Approves anything consequential	Nothing gets treated as resolved, and no journal entry gets generated, without an explicit human click

Why we didn't just let the AI "figure out" discrepancies directly: an LLM reading a spreadsheet and guessing why numbers don't match would be faster to build, but it would also be unverifiable — you can't audit a model's reasoning the way you can audit a line of Python. For a *financial reconciliation* tool specifically, being fast and occasionally wrong is worse than being slower and always explainable. That trade-off is the actual product decision at the center of this project.

4. Tech Stack (and why)

Layer	Choice	Reasoning
Backend	Python + FastAPI + SQLAlchemy	Deterministic matching/bridge logic needed to be simple, testable, and easy to read line-by-line — Python is the most natural fit for that, and FastAPI keeps the API layer lightweight

Layer	Choice	Reasoning
Database	SQLite	Fine for a demo scope; all the tables (settlement, bank, orders, matches, exceptions, approvals, audit log) are relational and don't need anything heavier
Frontend	React + Tailwind	Built separately, against a frozen API contract, once the backend was verified — dashboard-style UI for the bridge view, exception queue, and approval flow
AI layer (in-product)	Google Gemini API (Flash)	Used only for explaining exception packets in plain language and (later) natural-language querying over already-computed facts — never for calculation. Chosen for its free tier, sufficient for this project's small, cached call volume
Dev tooling (not in the product)	Claude Code + Cursor	Used to write the code itself — a separate use of AI from the in-product Gemini feature above

Why backend-first, not both at once: we deliberately did NOT build backend and frontend in parallel from day one. The data model changed materially on Day 1 (we discovered a whole missing table — see below), and building UI against a schema that was still shifting would have meant redoing UI work. We only started frontend once the API contract was frozen and tested against real (synthetic) data.

5. The Data Model — and the Real Bug We Caught Building It

This is worth knowing in detail, because it's a genuinely good story about why verification matters, not just a schema listing.

The tables, and what each represents

SettlementBatch — one settlement *cycle*: the totals Razorpay declares (gross, refunds, fees, tax, net) and the one bank transaction it's supposed to correspond to.

SettlementEntry — the individual order-level rows *within* a batch (15-20 per batch in our synthetic data).

BankTransaction — a row from the bank statement.

OrderRecord — the merchant's own internal record of what happened to an order (including the fee they *expected* to be charged and any refund they *believe* happened).

Match — records that a `SettlementBatch` was paired with a `BankTransaction`, with a confidence score and match type (exact/fuzzy).

ExceptionRecord — a flagged discrepancy: what's wrong, how much, and a suggested next action.

ApprovalLog — every human decision on an exception.

AuditEvent — an immutable log of every system action, exception, and approval. Never updated or deleted, by design.

Issue #1: The missing batch table

Our first draft of the schema (following the original project brief literally) had `SettlementEntry` as a flat, independent table with no concept of a "batch." But re-reading the actual problem statement: a merchant gets **one net settlement number** — one bank credit — covering *many* orders bundled together. Without a `SettlementBatch` table sitting above the individual order rows, there was no natural place to do the bridge math (which has to happen at the *batch* level, since that's the level at which one bank credit exists to compare against). We caught this before writing any logic on top of it and added the table — a good example of why we verified the schema by hand instead of trusting it on sight.

Issue #2: A one-to-many link that should have been one-to-one

`SettlementBatch.bank_transaction_id` originally created a relationship where, by default, one bank transaction could theoretically be linked to *multiple* settlement batches. That's not just untidy — it directly contradicts the real-world model: one bank credit settles exactly one batch, never two. We added a database-level `unique=True` constraint (not just a code-level check) specifically so that if a future bug in the matching engine ever tried to link two batches to the same bank transaction, the database itself would refuse it loudly instead of silently storing incorrect data.

Why this matters as a talking point: it shows the discipline of pushing correctness rules down into the schema itself wherever possible, rather than trusting application code to always remember to check.

Issue #3: A self-contradicting synthetic data row

While generating our "missing refund" injected error (a refund that exists in the settlement file but was never logged internally), we found `OrderRecord.status = "partially_refunded"` sitting next to `OrderRecord.refund_amount = 0.0` — internally contradictory on its own, without even looking at the settlement file. We fixed this so the `OrderRecord` reflects what the merchant's system *actually believes happened* (`status = "completed"`, refund = 0) — meaning the discrepancy is only visible by cross-referencing against the settlement file, which is the realistic version of this failure mode. This matters because it's exactly the kind of error a matching engine is supposed to catch — if it's obvious from looking at one table alone, it's not really testing anything.

Issue #4: The Match table's shape

We initially gave `Match` an entry-level foreign key (`settlement_entry_id`), following the original spec literally. But thinking through what a "match" actually represents in this domain: a `BankTransaction` is a lump-sum payment covering an *entire batch*, never a single order. There's no real scenario where one order gets individually matched to a bank transaction. So we simplified `Match` to be batch-level only (`settlement_batch_id`), removing a column that couldn't correspond to anything that actually happens — a small schema change, but one that makes the table's purpose unambiguous instead of leaving an unused nullable field a judge could reasonably ask "what's this for?"

6. Synthetic Dataset — What and Why

We use **synthetic data only**, never real Razorpay settlement data, deliberately: real merchant data can't be shared or independently checked by judges, so any accuracy claim built on it would be unverifiable.

Synthetic data with a documented `ground_truth.json` file lets anyone — including a judge — check our accuracy claim line by line.

Dataset shape (final): 4 settlement batches, ~15-20 orders each. Batch 1 is fully clean (zero errors). The other 3 batches carry exactly 4 injected error types between them:

Missing refund record (Batch 2) — a refund appears in the settlement file but was never logged in the merchant's own `OrderRecord`.

Wrong fee tier (Batch 2) — the fee actually charged doesn't match the fee the merchant expected (`OrderRecord.fee_amount`).

Duplicate entry (Batch 3) — the same order appears twice in the settlement file (a bookkeeping artifact, not a real bank-side discrepancy).

Timing mismatch (Batch 4) — the bank credit lands a few days after the settlement date, which a naive same-day matcher would fail to catch.

Why 4 batches, not 3: timing mismatch and duplicate entry were originally both planted in the same batch, but they turned out to be structurally incompatible signals — timing difference is only classified when the batch's internal math is perfectly clean (it's *purely* a date issue), while duplicate detection specifically requires the internal math to *not* be clean. One batch can't honestly demonstrate both at once, so we split them into separate batches so each of the 4 error types has its own clean, independently verifiable demonstration.

The dataset generator uses a fixed random seed, so regenerating it produces byte-identical data every time — every specific number cited in this document and in the demo will be reproducible on request.

These 4 map to real reconciliation failure modes accountants actually encounter — they weren't picked arbitrarily for the demo.

7. Matching Engine + Bridge — What It Does, and a Real Bug We Fixed

What it does

For each `SettlementBatch`: find the `BankTransaction` it corresponds to (by amount and date, with a tolerance window), assign a confidence score (1.0 for exact same-day match, 0.85 for a fuzzy match where dates differ by a few days), then compute the bridge: sum the batch's entries (`gross - refund - fee - tax`), compare that sum to the batch's *declared* net total, and separately compare the declared net total to the actual bank amount. Any gap in either comparison is a signal something needs investigating.

The bug we found and fixed

Our first version of `ingest.py` calculated a batch's `total_net` by **summing its `SettlementEntry` rows** — which sounds reasonable, until you remember one batch has a *deliberately duplicated* entry. That duplication silently got double-counted into the derived total, which broke two things at once: the matcher couldn't find the right bank transaction anymore (the derived total was now ~₹8,385 too high),

and the bridge's "does this add up" check was comparing against a number that was already wrong before any real discrepancy was even introduced.

The fix: we added a 4th data source — `settlement_batches.csv` — representing the totals Razorpay *actually declares* for each batch, generated independently of the entry-level rows. `ingest.py` now reads these declared totals directly instead of deriving them by re-summing entries. This isn't a workaround; it's actually a more accurate model of the real problem — a merchant doesn't derive their settlement total by manually adding every order line, Razorpay states a total, and the job is to verify that stated total is correct.

Why this bug is a good talking point, not something to hide: it demonstrates that our matching/bridge logic is actually sensitive to the data it's given — a "does it look plausible" demo that never breaks anything wouldn't prove much. This one broke, we found exactly why, and fixing it produced a cleaner signal: the duplicate now shows up specifically as an **internal bookkeeping mismatch** (entries don't sum to the declared total) while the batch still **matches the bank correctly** (declared total does equal what the bank paid). That distinction — real external variance vs. internal bookkeeping artifact — is exactly the kind of nuance the exception engine (Day 3) needs to classify things correctly, and it fell directly out of fixing this bug rather than being designed in advance.

8. Exception Engine — Classification Logic (confirmed working, all 4 error types verified independently)

Every batch is evaluated against these rules. Critically, **a batch can receive more than one exception record**, and the refund/fee checks run independently of the bank-side variance checks — more on why below.

Signal	Classification	Blocks the "reconciled" banner?	Needs human approval?
No bank transaction found	UNMATCHED_BATCH	Yes	Yes
Entries don't sum to declared total, but declared total matches the bank	DUPLICATE_ENTRY	No	Yes (needs confirmation to remove the duplicate)
A refund was expected in settlement but not logged internally (checked on every batch, regardless of bank-side variance)	MISSING_REFUND_RECORD	Yes	Yes
The actual fee doesn't match the expected fee (checked on every batch, regardless of bank-side variance)	FEE_TIER_MISMATCH	Yes	Yes
Fuzzy match (date differs) but every total lines up exactly, and no other exception applies	TIMING_DIFFERENCE	No	No — informational only
Declared total $\neq$ bank amount, but none of the above patterns explain it	UNEXPLAINED_VARIANCE	Yes	Yes

A real design correction worth knowing well: why refund/fee checks can't be gated behind bank-side variance

Our first version only checked for `MISSING_REFUND_RECORD` / `FEE_TIER_MISMATCH` when the declared total didn't match the bank amount. That gate can **never fire** for these two errors, structurally — and understanding why is one of the sharper insights in this whole project.

Razorpay computes its declared net total *correctly* — it genuinely does subtract the real refund and charge the real fee. The bank then pays out exactly that correct declared amount. So the bank-side variance is **zero even when these errors exist**, because nothing about Razorpay's or the bank's numbers is wrong. What's wrong is quieter and worse: the *merchant's own internal records* were never updated to reflect that refund or that fee — and since the money that landed in the bank matches perfectly, nothing about the bank-side numbers would ever tip anyone off. This is a stronger version of the "silent leak" problem than a rupee-level variance is, precisely because there's no variance to notice at all — you can only catch it by directly cross-referencing the settlement file against the merchant's own order records, which is exactly the manual, tedious comparison most merchants skip. We fixed this by decoupling the check: every batch gets scanned for refund/fee mismatches against `OrderRecord` regardless of whether the bank number matches, and a batch can be `is_reconciled: false` even when `variance == 0`, as long as any exception — of any kind — is still open. That's Principle 1 enforced literally: a batch isn't reconciled just because the bank number checks out; it's reconciled when every open question is closed.

Why timing differences still get logged even though nothing's wrong financially: logging it as a low-severity, non-blocking, no-approval-needed record keeps the "4/4 error types caught" claim honest and complete, rather than silently treating "processing lag" as invisible.

Why suggested actions come from a fixed table, never generated by AI: a suggested next action ("add refund entry and verify against the gateway log") is a business rule, not a judgment call an AI should be inventing per-case. The AI's job later is only to explain *why* a given exception was classified this way, in plain language, using the already-computed classification and evidence — never to decide the classification itself.

Confirmed verification results (all 4 independently demonstrated)

Batch	Classification(s)	Result
1	None	Fully clean, <code>is_reconciled: true</code>
2	<code>MISSING_REFUND_RECORD</code> (₹5,786.83) + <code>FEE_TIER_MISMATCH</code> (₹3.49)	Both fire as separate exception records on the same batch; <code>variance: 0.0</code> but <code>is_reconciled: false</code>
3	<code>DUPLICATE_ENTRY</code>	<code>blocks_reconciliation: false</code> , but still <code>is_reconciled: false</code> until approved
4	<code>TIMING_DIFFERENCE</code>	<code>requires_approval: false</code> , <code>is_reconciled: true</code> — a fuzzy match with no other issue doesn't block reconciliation

9. Audit Trail — Why Immutability Is a Design Rule, Not Just a Nice-to-Have

Every match created and every exception raised writes an `AuditEvent` row (`actor`, `action`, `before_state`, `after_state`, `timestamp`). We deliberately never write update or delete logic for this table anywhere in the codebase — not because SQLite can't support it, but because an audit trail that can be edited after the fact isn't actually an audit trail. If a judge asks "can this be tampered with," the honest answer needs to be "there's no code path that modifies a past audit record — only new records get appended."

We actually proved this during our own testing, which is a good concrete story: while testing the approval endpoint, one exception got marked "approved" as part of a test call, not a real decision. Rather than being able to quietly erase that, the append-only design meant the only way to get back to a clean state was to add *new* records on top — the test approval is still sitting in the audit log, timestamped, clearly attributed to that test, sandwiched between two system pipeline runs. Nothing was hidden or rewritten. That's not a hypothetical guarantee — it's something we watched happen to our own data.

9a. The API Layer — What Cursor Builds Against

Six FastAPI endpoints wrap the backend logic, all tested against real (synthetic) data before any frontend

work started:

Endpoint	Purpose
<code>GET /batches</code>	All settlement batches with computed fields: totals, matched bank amount, confidence, <code>is_reconciled</code> , <code>variance</code>
<code>GET /batches/{id}</code>	One batch's summary plus its full list of order-level entries
<code>GET /batches/{id}/exceptions</code>	All exception records for a batch — classification, amount, suggested action, status
<code>GET /batches/{id}/evidence</code>	Resolves an exception's linked evidence IDs into the actual source rows — this is what the drill-down/approval-brief UI renders
<code>POST /exceptions/{id}/approve</code>	Records a human decision — writes an approval log entry and an audit event, updates only the exception's status, never touches any bridge/match/total number
<code>GET /audit-trail</code>	Full paginated event history

Why the API was built and fully tested before any UI work started: the data model changed materially multiple times during Days 1-3 (a missing batch table, a schema shape correction, a new foreign key discovered while building this very API). Building frontend against an API that might still change would mean redoing UI work every time the backend shifted. Freezing the contract only once it was tested against real data — including edge cases like "variance is zero but still not reconciled" — meant the frontend could be built once, correctly, instead of iteratively chasing a moving backend.

A concrete example of why testing the real JSON mattered, not just the endpoint code: building the `/exceptions` and `/evidence` endpoints surfaced a genuine schema gap — `ExceptionRecord` had no way to look up which batch it belonged to. This wasn't a new requirement; it's something the exception engine always needed, but it only became visible once something actually needed to query by batch. Better to find that while the API was still read-only and being tested by hand than after a UI had been built assuming it already worked.

10. What's Simulated vs. What's Real (be exact about this)

Component	Status
Matching engine	Real — running deterministic logic on real (synthetic) data
Bridge calculation	Real — same
Exception classification	Real — rule-based, not AI
Audit trail	Real — actual database writes

Component	Status
AI explanation of exceptions	Real AI calls (Google Gemini), but only ever narrating already-computed facts, with every number in the response programmatically verified against the source data before being shown
Journal entry posting	Simulated — clearly labeled as "Simulated — requires ERP integration," never actually posts anywhere
Underlying settlement/bank/order data	Synthetic — deliberately, so accuracy claims are checkable by anyone

Being exact and upfront about this list is a strength, not a weakness, in front of judges — it shows you know precisely where the line between "built and working" and "future integration work" sits.

11. Anticipated Judge Questions — Prepared Answers

"Does the AI ever touch the arithmetic?" No. The matching, bridge, and exception logic are all plain Python — happy to pull up `app/matching.py` or `app/bridge.py` directly and walk through it line by line.

"How do you know your accuracy numbers aren't cherry-picked?" We test against a synthetic dataset with a documented `ground_truth.json` listing exactly which errors were planted, where, and what the expected vs. actual values are. Anyone — including you — can check our detection results against that file directly.

"What happens if the AI's explanation or classification is wrong?" The AI only ever explains a classification the deterministic code already made — it doesn't get to change it. A human reviews the evidence and either approves or rejects; nothing about the underlying exception record changes until they do. We also don't just trust the AI's wording blindly: every number in an AI-generated explanation is programmatically checked against the actual source data before it's ever shown. If the AI's text contains any figure that doesn't match, that response is discarded automatically and replaced with a safe, templated explanation instead — so an ungrounded number can never reach the screen.

"Why Gemini and not Claude for the AI explanation feature, if you built the app with Claude Code?" Those are two separate uses of AI — Claude Code and Cursor were development tools we used to write the code; Gemini is what the finished product itself calls when it explains an exception to a user. We chose Gemini for that specific feature because its free tier comfortably covers this project's actual usage (a handful of short, cached calls), and the safety design — never let AI touch arithmetic, verify every number before display — doesn't depend on which provider is behind it. The architecture principle is provider-agnostic; the choice of Gemini here was a practical one.

"Does this scale beyond Razorpay's settlement format specifically?" Right now, honestly, no — it's built against Razorpay's settlement structure specifically. Generalizing to other payment processors would mean writing new ingestion/parsing logic for their file formats, but the matching, bridge, and exception-classification logic underneath would carry over largely unchanged, since that logic works on the internal data model, not the raw file format.

"What was the hardest part of getting the math right?" Genuinely: getting the *duplicate entry* case right. Our first version accidentally let a duplicated settlement row double-count itself into the declared batch total, which broke the matcher and hid the real bug. Fixing it meant introducing a fourth data source — Razorpay's *declared* batch totals, independent of the entry-level rows — so the system verifies entries against a declared total instead of just re-deriving one and trusting its own math.

"What's the scariest failure mode you found while building this?" Not a rupee-level variance — those at least show up somewhere. The scarier one is a batch where the bank amount matches Razorpay's declared total *perfectly* (`variance: 0.0`), but the merchant's own internal order record was never updated with a refund or fee that actually happened. Nothing about the bank-side numbers looks wrong, so a merchant checking "does the deposit match what I expected" would find nothing. We only catch it because we don't stop checking once the bank number matches — we still cross-reference every batch's entries against the merchant's own records, every time.

"Can the audit trail be faked or edited?" No code path exists to update or delete a past `AuditEvent` row — only new rows get appended, ever. We actually saw this in practice: a test approval action during our own development left a permanent, clearly timestamped record in the log, and the only way to move forward was to add new events on top of it, not erase it.

12. Build Methodology (be upfront about this — it's not a weakness)

This was built solo, with AI writing the code (Claude Code for backend, Cursor for frontend) under close human review at every step. Note this is a separate use of AI from the product's own in-app AI feature (Google Gemini, used only to explain exceptions in plain language) — the tools that built the app and the AI inside the app are not the same thing. The discipline that made this workable: **verify each deterministic component against known ground truth before building anything on top of it**. Real issues — a missing database table, an incorrect one-to-many relationship, a self-contradicting data row, a schema that couldn't represent the real-world relationship it needed to, a data-derivation bug that silently double-counted a duplicate, a classification rule that could structurally never fire, two error types that couldn't coexist in one batch, and a missing foreign key discovered only once the API needed it — were all caught specifically *because* each layer was checked by hand against what was actually expected before moving to the next one, rather than trusting generated code on sight.

If asked "did you write this code yourself" — the honest answer is that AI generated it, decisions and verification were yours at every step, and you can explain *why* every non-trivial decision was made, including several the AI got wrong on the first attempt and you corrected.

This document will be updated as Day 4 (frontend, Cursor) and Day 5 (final features, demo prep) complete. Sections describing the UI do not exist yet — they'll be added once Day 4 work is verified.